UADE · Paradigma Orientado a Objetos · 2026

Trabajo Práctico Obligatorio

Boletería

Sistema de venta de entradas para sedes de espectáculos

Integrantes · Grupo 3

Apellido y nombre	Legajo
Aded, Fermín	1201530
Ávila, Tomás	1210748
Vázquez, Daniel	1220473
Villa Abrille, Malena	1193462

Noviembre 2026

Índice

1. Propuesta de negocio
2. Flujo de compra
3. Diagrama de clases
4. Clases del sistema
5. Patrones y buenas prácticas
6. Manejo de excepciones
7. Persistencia
8. Estructura de archivos del código
9. Interfaz gráfica

1. Propuesta de negocio

El problema

Las sedes de espectáculos de gran escala, como la Esfera de Las Vegas, venden miles de entradas por función y cada ubicación tiene un precio distinto según el piso y la cercanía al escenario. Cuando la venta se hace a mano o con herramientas genéricas aparecen problemas que le cuestan dinero a la sede y paciencia al público: el mismo asiento se vende dos veces, quedan asientos sueltos que después nadie compra, se pierden ventas porque alguien "retiene" lugares sin pagar, las preventas no se controlan y las devoluciones se resuelven caso por caso.

La solución

Boletería es una aplicación de escritorio en Java que resuelve la venta de entradas de punta a punta: desde la configuración de la sede y sus sectores hasta la factura y la devolución. El sistema calcula el precio de cada ubicación, reserva los asientos elegidos durante 5 minutos mientras el cliente paga, valida automáticamente las reglas de venta y deja todo registrado en archivos.

En una frase: una boletería que nunca vende dos veces el mismo asiento, no deja asientos sueltos, libera sola los lugares que nadie pagó y gestiona las devoluciones con reglas claras.

Usuarios

- Administración de la sede: da de alta las sedes con sus sectores (numerados o generales), piso y distancia al escenario.
- Boletería: vende entradas a los clientes registrados, aplica promociones, emite los comprobantes y procesa las devoluciones.
- Clientes: se registran con email y contraseña, compran y pueden devolver sus entradas dentro del plazo.

Funcionalidades

Funcionalidad	Descripción
Gestión de sedes (ABM)	Alta, baja, modificación y consulta de sedes y sus sectores. Los sectores numerados se generan por filas (A, B, C…) y asientos por fila; los generales tienen capacidad.
Precio por ubicación	Precio de la entrada = precio base de la función × multiplicador del sector. El multiplicador crece cuanto más cerca del escenario y más bajo es el piso; en un sector general (de pie) se aplica un 20 % menos.
Venta de entradas	Mapa de asientos con el escenario como referencia. El cliente elige sus lugares, se reservan por 5 minutos y se paga.
Reglas de venta	No se permite dejar un asiento libre aislado. Durante la preventa solo se compra con el código de acceso.
Promociones	Descuento porcentual sobre las entradas al ingresar un código promocional (por ejemplo, de un banco o del club de fans).
Factura	Al confirmar el pago se emite un comprobante con el detalle, el cargo por servicio y el medio de pago, y se guarda como archivo de texto.
Devoluciones	Se puede devolver una entrada pagada hasta 48 horas antes de la función, o en cualquier momento si la función se cancela. Se reintegra lo pagado por la entrada, sin el cargo por servicio.
Cuentas	Registro con DNI, nombre, email y contraseña, e inicio de sesión. La contraseña se guarda cifrada (hash SHA-256).

Alcance

Queda fuera del sistema: la producción del show (proyecciones, efectos), el cobro real a través de pasarelas de pago y la reventa entre particulares. Por devolución del docente en la Entrega 1 se quitaron el estacionamiento, las experiencias inmersivas, el precio dinámico y los tipos de cargo, para centrar el proyecto en la venta de entradas. En la Entrega 2 se simplificó además el modelo de 33 a 26 clases, unificando las que no tenían comportamiento propio (por ejemplo, `Cuenta` dentro de `Cliente` y `Pago` dentro de `Compra`) sin perder funcionalidad.

2. Flujo de compra

Este es el recorrido completo de una compra, tal como lo implementa el sistema:

1. Búsqueda: se elige el evento y la función. Antes de mostrar el mapa se liberan las reservas vencidas.
2. Selección: se marcan los asientos (o la cantidad, en un sector general) y, si la función está en preventa, se ingresa el código.
3. Validación: la boletería recorre las reglas de venta. Si alguna no se cumple, se informa el motivo y no se reserva nada.
4. Reserva: se crea la compra en estado RESERVADA con vencimiento a 5 minutos. Las entradas ocupan su lugar y arranca la cuenta regresiva.
5. Adicionales: se puede sumar el paquete VIP y un código promocional. El resumen muestra subtotal, descuento, cargo por servicio (12 %) y total.
6. Pago: si pasaron más de 5 minutos, la compra vence y los asientos se liberan. Si no, la compra registra el medio de pago y el número de operación, las entradas pasan a PAGADA y se emite la factura.
7. Devolución (posterior): con el código de la entrada se verifica el plazo de 48 horas, se reintegra lo pagado por ella y el asiento vuelve a la venta.

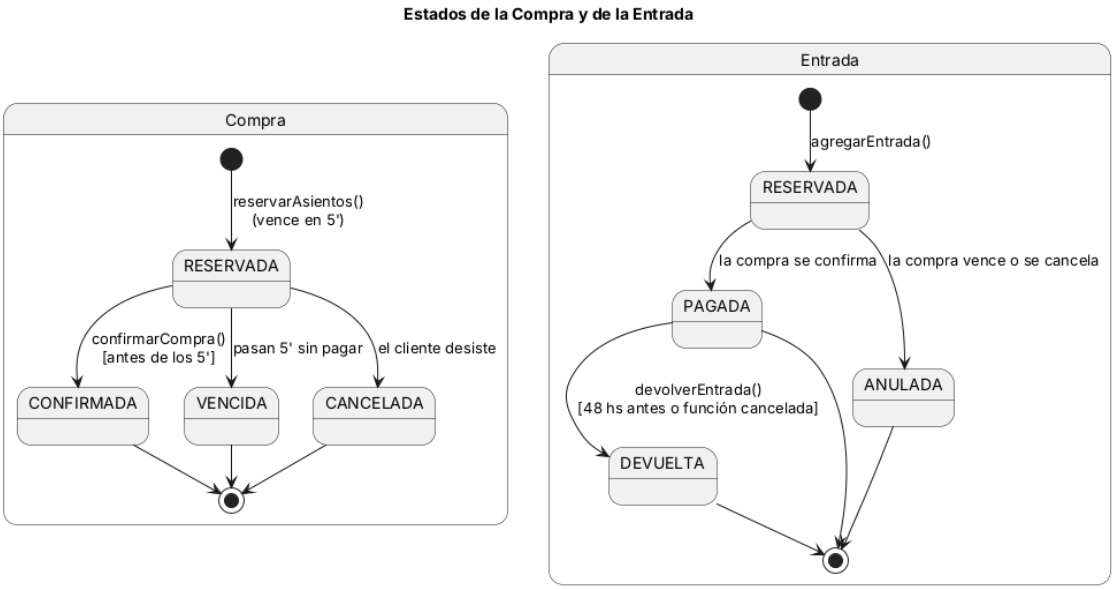


Figura 1. Estados de la Compra y de la Entrada.

Reglas de negocio

Regla	Cómo funciona
Reserva de 5 minutos	Constante `Compra.MINUTOS_RESERVA`. Una compra RESERVADA vence si no se paga a tiempo y sus entradas pasan a ANULADA.
No dejar un asiento suelto	`NoDejarAsientoSuelto` rechaza una selección que deje un asiento libre entre dos ocupados (o entre uno ocupado y el borde de la fila).
Preventa	`RequiereCodigoPreventa` exige el código de acceso mientras la preventa de la función está vigente.
Disponibilidad	La administra cada `Funcion`: el mismo asiento físico se vende por separado en cada función.
Cargo y promoción	El cargo por servicio (`Compra.CARGO_SERVICIO`, 12 %) se calcula sobre el subtotal ya descontado. Una compra admite una sola promoción.
Devolución	Solo entradas PAGADAS, hasta 48 horas antes (`Devolucion.HORAS_LIMITE`) o con la función cancelada. Se reintegra lo que se pagó por la entrada, con su parte del descuento y sin el cargo.

3. Diagrama de clases

Versión alineada con el código. Se adjunta en PNG y en PlantUML en `diagramas/clases/diagrama_clases_entrega2`; incluye las tres capas de MVC (vista, controlador y modelo).

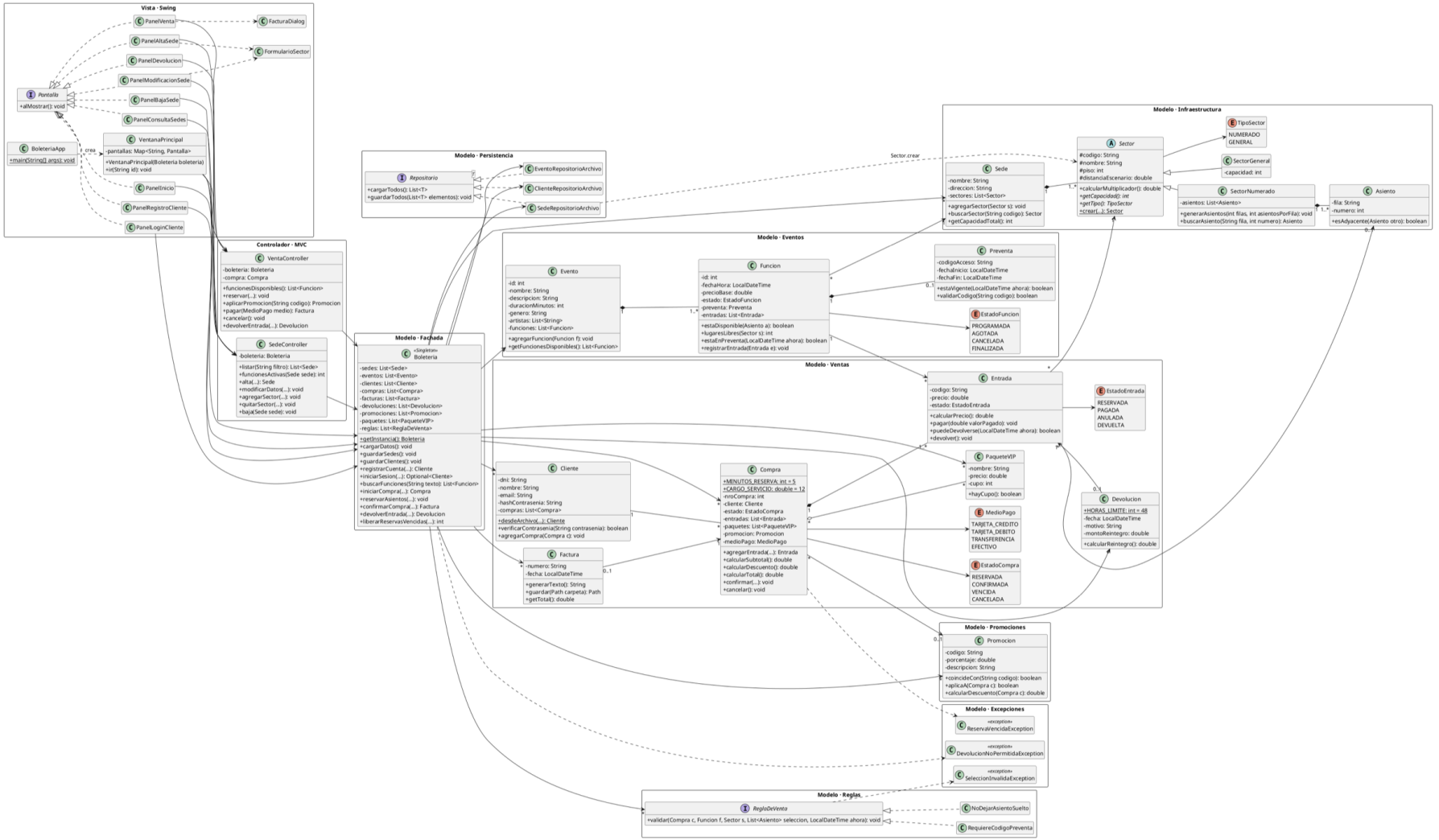


Figura 2. Diagrama de clases (alineado con el código de la Entrega 2).

4. Clases del sistema

El modelo tiene 26 clases e interfaces (2 interfaces y 3 excepciones entre ellas), sin contar las pantallas, y 5 enumeraciones, organizadas en paquetes:

Clase	Tipo	Responsabilidad
Infraestructura		
Sede	Clase	Lugar físico del espectáculo. Contiene sus sectores y su capacidad total.
Sector	Abstracta	Código, nombre, piso y distancia al escenario. Calcula el multiplicador de precio. Su método estático `crear` fabrica el sector que corresponda según el tipo.
SectorNumerado	Clase	Sector con asientos por fila. Genera los asientos y los busca por fila y número.
SectorGeneral	Clase	Sector de pie definido por su capacidad. Redefine el multiplicador (−20 %).
Asiento	Clase	Fila y número. Sabe si es adyacente a otro asiento.
Eventos		
Evento	Clase	Nombre, descripción, duración, género y artistas. Agrupa sus funciones.
Funcion	Clase	Fecha de un evento en una sede. Controla disponibilidad, ocupación y estado.
Preventa	Clase	Código de acceso y período de vigencia.
Ventas		
Cliente	Clase	DNI, nombre, email, hash de la contraseña e historial de compras. Verifica el inicio de sesión.
Compra	Clase	Reserva de 5 minutos, medio de pago y número de operación. Crea sus entradas y calcula subtotal, descuento, cargo por servicio y total.
Entrada	Clase	Función, sector, asiento, precio, valor pagado y estado. Sabe si se puede devolver.
PaqueteVIP	Clase	Adicional con beneficios, precio y cupo.
Factura	Clase	Comprobante de una compra pagada. Genera su texto y lo guarda como archivo.
Devolucion	Clase	Entrada devuelta, motivo y monto a reintegrar.
Promocion	Clase	Código y porcentaje de descuento sobre las entradas.
Reglas, control y persistencia		
ReglaDeVenta	Interfaz	Regla que se valida antes de reservar.
NoDejarAsientoSuelto	Clase	Impide dejar un asiento libre aislado.
RequiereCodigoPreventa	Clase	Exige el código durante la preventa.

Clase	Tipo	Responsabilidad
Boleteria	Singleton	Fachada del modelo: punto de entrada a todas las operaciones del sistema y guarda el cliente con sesión iniciada.
Repositorio<T>	Interfaz	Cargar y guardar una lista de objetos.
SedeRepositorioArchivo	Clase	Guarda sedes y sectores en archivos de texto.
ClienteRepositorioArchivo	Clase	Guarda los clientes, con el hash de su contraseña, en un archivo de texto.
EventoRepositorioArchivo	Clase	Guarda los eventos y sus funciones (fecha, precio, estado y preventa) en archivos de texto.
Excepciones del negocio		
SeleccionInvalidaException	Excepción	La selección no cumple una regla (ocupado, suelto, preventa).
ReservaVencidaException	Excepción	Se intentó pagar una reserva vencida.
DevolucionNoPermitidaException	Excepción	La entrada no está en condiciones de devolverse.

Enumeraciones: `EstadoFuncion`, `EstadoCompra`, `EstadoEntrada`, `MedioPago` y `TipoSector`.

Relaciones principales

- Herencia: `Sector` → `SectorNumerado` y `SectorGeneral`.
- Realización: `ReglaDeVenta` (`NoDejarAsientoSuelto`, `RequiereCodigoPreventa`) y `Repositorio<T>` (`SedeRepositorioArchivo`, `ClienteRepositorioArchivo`, `EventoRepositorioArchivo`).
- Composición: `Sede` ◆ `Sector`, `SectorNumerado` ◆ `Asiento`, `Evento` ◆ `Funcion`, `Funcion` ◆ `Preventa`, `Compra` ◆ `Entrada`.
- Agregación: `Compra` ◇ `PaqueteVIP`.
- Asociación: `Funcion` → `Sede`; `Entrada` → `Sector` y `Asiento` (0..1); `Compra` → `Promocion` (0..1); `Factura` → `Compra`; `Devolucion` → `Entrada`; `Cliente` — `Compra` (bidireccional).

5. Patrones y buenas prácticas

Patrón / principio	Dónde se aplica
Singleton	`Boleteria.getInstancia()`: una única boletería para todo el sistema.
Strategy	`ReglaDeVenta`: la boletería valida una lista de reglas sin conocer cada implementación. Para sumar una regla se crea una clase nueva.
Factory	`Sector.crear(...)` (método de fábrica estático) crea el sector numerado o general; lo usan las pantallas y el repositorio que lee los archivos. Es el único lugar con un `switch` por tipo.
Repository	`Repositorio<T>` separa el dominio de la forma de guardar los datos (hoy archivos; mañana podría ser una base de datos).
MVC	Las pantallas Swing (paquete `vista`) delegan en `SedeController` y `VentaController` (paquete `controlador`), que operan sobre `Boleteria` y el dominio (paquete `modelo`).

Patrón / principio	Dónde se aplica
GRASP Controlador	`Boleteria` recibe las operaciones del sistema y las delega en las clases del dominio.
GRASP Creador	`Compra` crea sus `Entrada`; `Boleteria` crea la `Factura` a partir de una compra pagada; `SectorNumerado` genera sus `Asiento`.
GRASP Experto	`Funcion` sabe qué asientos están libres; `Sector` calcula su multiplicador; `Compra` calcula su total.
Polimorfismo / Liskov	`SectorNumerado` y `SectorGeneral` redefinen `getCapacidad()` y `calcularMultiplicador()` y se usan donde se espera un `Sector`.
Abierto / cerrado	Nuevas reglas de venta, promociones o repositorios se agregan sin modificar el código existente.
Encapsulamiento	Atributos privados, validaciones en los constructores y listas expuestas como no modificables.

6. Manejo de excepciones

Las excepciones propias del negocio son verificadas (extienden `Exception`), así que quien llama está obligado a tratarlas:

Excepción	Cuándo se lanza
SeleccionInvalidaException	Al reservar: asiento ocupado, selección vacía, asiento suelto o falta el código de preventa.
ReservaVencidaException	Al confirmar una compra cuya reserva superó los 5 minutos.
DevolucionNoPermitidaException	Al devolver una entrada no pagada, inexistente o fuera del plazo de 48 horas.

Un inicio de sesión fallido no es una excepción: `Boleteria.iniciarSesion` devuelve un `Optional` vacío, porque es un resultado esperable y no un error. Los datos inválidos (nombre vacío, DNI mal formado, código promocional inexistente) se rechazan con `IllegalArgumentException`, y las operaciones que no corresponden al estado del objeto (dar de baja una sede con funciones, agregar algo a una compra ya pagada) con `IllegalStateException`.

Bloques try/catch principales:

- `PanelVenta`: captura `SeleccionInvalidaException` e `IllegalStateException` al reservar y `ReservaVencidaException` al pagar, y muestra el motivo en pantalla.
- `VentaController.reservar`: si una regla falla, cancela la compra (libera asientos y cupo VIP) y relanza la excepción para que la vista la muestre.
- `PanelDevolucion`: captura `DevolucionNoPermitidaException` al verificar y al devolver una entrada.
- `PanelRegistroCliente` y `PanelLoginCliente`: capturan `IllegalArgumentException` (datos inválidos o email repetido) e `IOException` al guardar, y avisan en pantalla.
- `PanelAltaSede`, `PanelModificacionSede` y `PanelBajaSede`: datos inválidos, sede con funciones programadas e `IOException` al guardar.
- `BoleteriaApp`: error al leer los archivos al iniciar la aplicación (muestra un diálogo y no abre la ventana).
- `Boleteria.confirmarCompra`: si falla guardar la factura, la venta no se pierde.

- `SedeRepositorioArchivo`: convierte un registro mal formado en una `IOException` con el número de línea.
- `EventoRepositorioArchivo`: hace lo mismo con `funciones.txt`, y rechaza una función cuya sede no existe.
- `Cliente.hashear`: convierte `NoSuchAlgorithmException` en `IllegalStateException` (SHA-256 siempre existe en Java).

7. Persistencia

Los datos se guardan en archivos de texto dentro de la carpeta `datos/`, un registro por línea y campos separados por "|". Si la carpeta no existe, se crea con datos de ejemplo.

Archivo	Formato
sedes.txt	`nombre|dirección`
sectores.txt	`sede|NUM|código|nombre|piso|distancia|filasXasientos` o `sede|GEN|código|nombre|piso|distancia|capacidad`
clientes.txt	`dni|nombre|email|hashContraseña`
eventos.txt	`id|nombre|descripción|duraciónMin|género|artistas`
funciones.txt	`idEvento|idFunción|sede|fechaHora|precioBase|estado|códigoPreventa|inicioPreventa|finPreventa`
factura_seq.txt	Número de la próxima factura. Si falta, se infiere de los comprobantes guardados.
facturas/FAC-*.txt	Un comprobante por compra confirmada, escrito por `Factura.guardar`.

Las sedes, los sectores, los clientes, los eventos y las funciones se cargan al iniciar y se guardan en cada alta, baja o modificación. La primera vez que se ejecuta, `DatosIniciales` crea los datos de ejemplo (con funciones a partir de la fecha del día) y los guarda en estos archivos.

8. Estructura de archivos del código

El proyecto es Maven (Java 17). Todo el código está en el paquete `ar.edu.uade.poo.grupo3.tpo`, dividido en las tres capas de MVC:

Paquete	Contenido
vista	`BoleteriaApp` (inicio), `VentanaPrincipal` (menú y `CardLayout`), las pantallas `PanelInicio`, `PanelConsultaSedes`, `PanelAltaSede`, `PanelModificacionSede`, `PanelBajaSede`, `PanelRegistroCliente`, `PanelLoginCliente`, `PanelVenta` y `PanelDevolucion`, el diálogo `FacturaDialog`, `FormularioSector`, `Estilo`, `Aviso` y la interfaz `Pantalla`. Solo arman la pantalla y muestran mensajes.
controlador	`SedeController` (ABM de sedes) y `VentaController` (compra en curso, reloj de la reserva y devoluciones). Reciben lo que hace el usuario y se lo piden al modelo.
modelo	`Boleteria` (Singleton, fachada del modelo) y los subpaquetes del dominio.
modelo.infraestructura	`Sede`, `Sector` (con la fábrica `Sector.crear`), `SectorNumerado`, `SectorGeneral`, `Asiento`.

Paquete	Contenido
modelo.eventos	`Evento`, `Funcion`, `Preventa`.
modelo.ventas	`Cliente`, `Compra`, `Entrada`, `PaqueteVIP`, `Factura`, `Devolucion`.
modelo.promociones	`Promocion`.
modelo.reglas	`ReglaDeVenta`, `NoDejarAsientoSuelto`, `RequiereCodigoPreventa`.
modelo.excepcion	Las tres excepciones del negocio.
modelo.enums	`EstadoFuncion`, `EstadoCompra`, `EstadoEntrada`, `MedioPago`, `TipoSector`.
modelo.persistencia	`Repositorio<T>`, `SedeRepositorioArchivo`, `ClienteRepositorioArchivo`, `EventoRepositorioArchivo`.
(raíz)	`DatosIniciales` (configuración y datos de ejemplo) y `DemoFlujoCompra` (flujo completo por consola).

Cómo ejecutarlo

- Aplicación: `java -jar Boleteria-TPO.jar` o `mvn compile exec:java` desde la carpeta `boleteria/`. En Mac también se puede hacer doble click en `Ejecutar Boleteria.command`.
- Demo por consola: `mvn compile exec:java -Dexec.mainClass=ar.edu.uade.poo.grupo3.tpo.DemoFlujoCompra`
- Datos de prueba: cuentas `ana@mail.com`, `bruno@mail.com` y `carla@mail.com` (contraseña `boleteria123`); preventa `ORBITA26`; promociones `FAN10` (10 %) y `BANCO20` (20 %).

9. Interfaz gráfica

La aplicación tiene una ventana principal con un menú (Inicio / Sedes / Clientes / Ventas / Ayuda, con atajos de teclado) que navega entre las pantallas con un `CardLayout`. Cada pantalla muestra los errores y confirmaciones en un aviso dentro de la misma pantalla. Los diálogos se reservan para la confirmación de la baja de una sede, el error de arranque, el comprobante de la compra y "Acerca de". Para comprar hay que iniciar sesión (Clientes → Iniciar sesión): la pantalla de venta muestra el cliente de la sesión y solo habilita Reservar cuando hay una sesión activa.

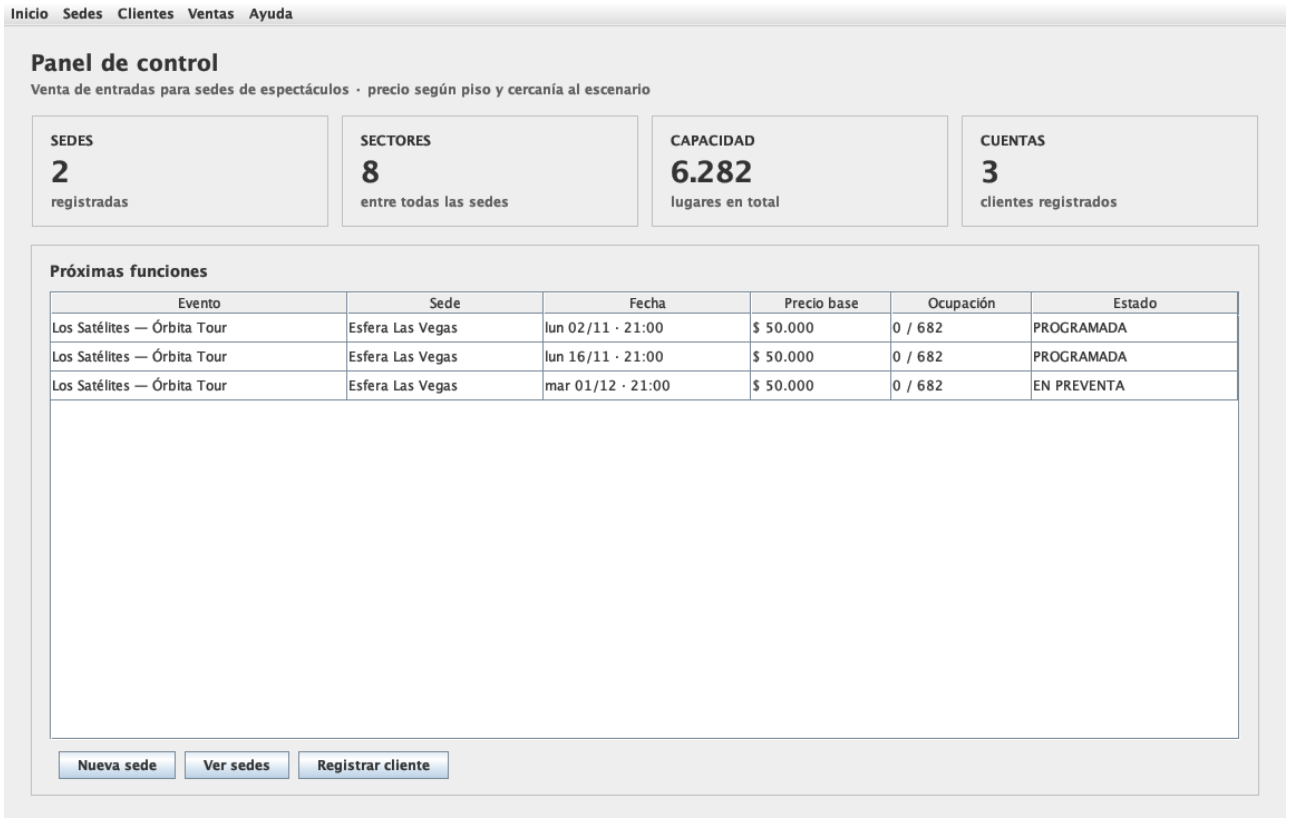


Figura 3. Inicio: métricas del sistema, próximas funciones y accesos directos (nueva sede, ver sedes, registrar cliente).

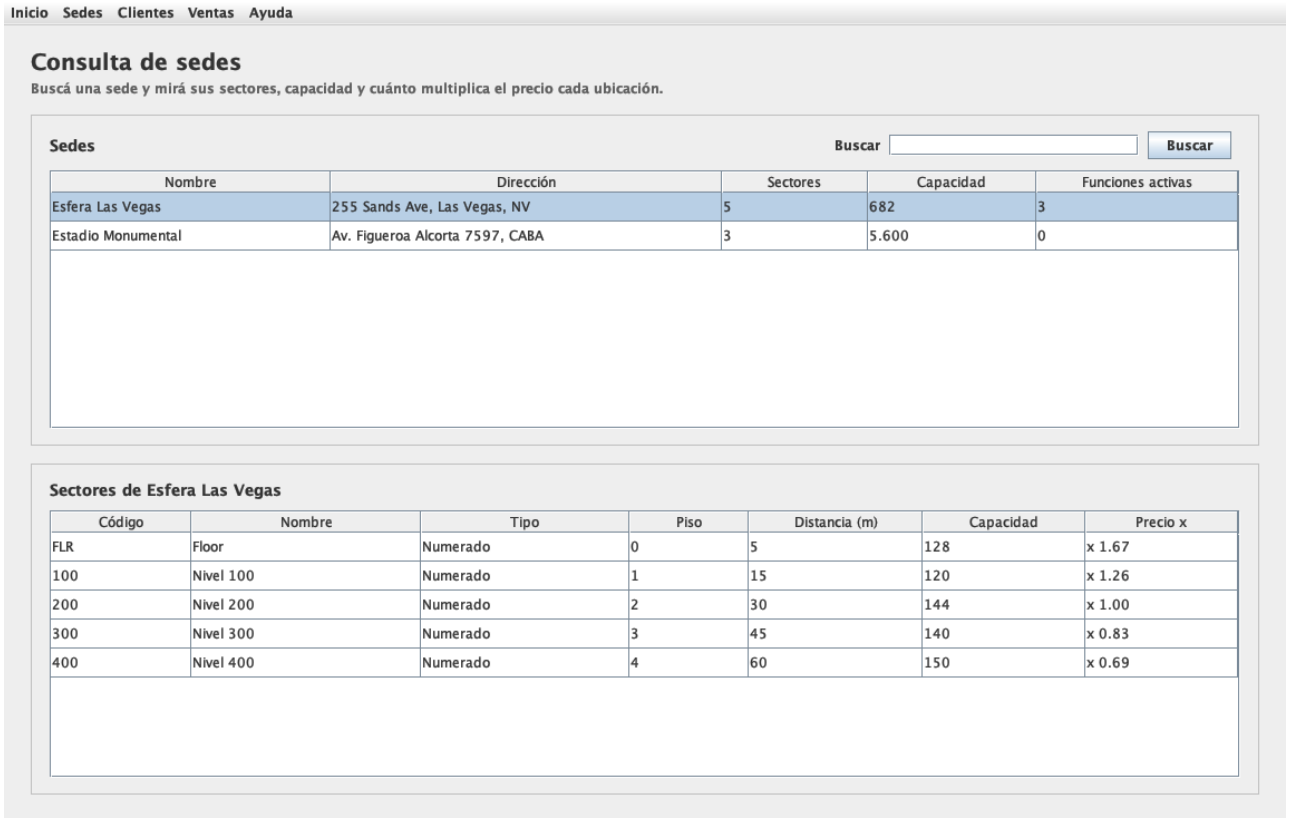


Figura 4. Consulta de sedes con buscador y detalle de sectores y su multiplicador de precio.

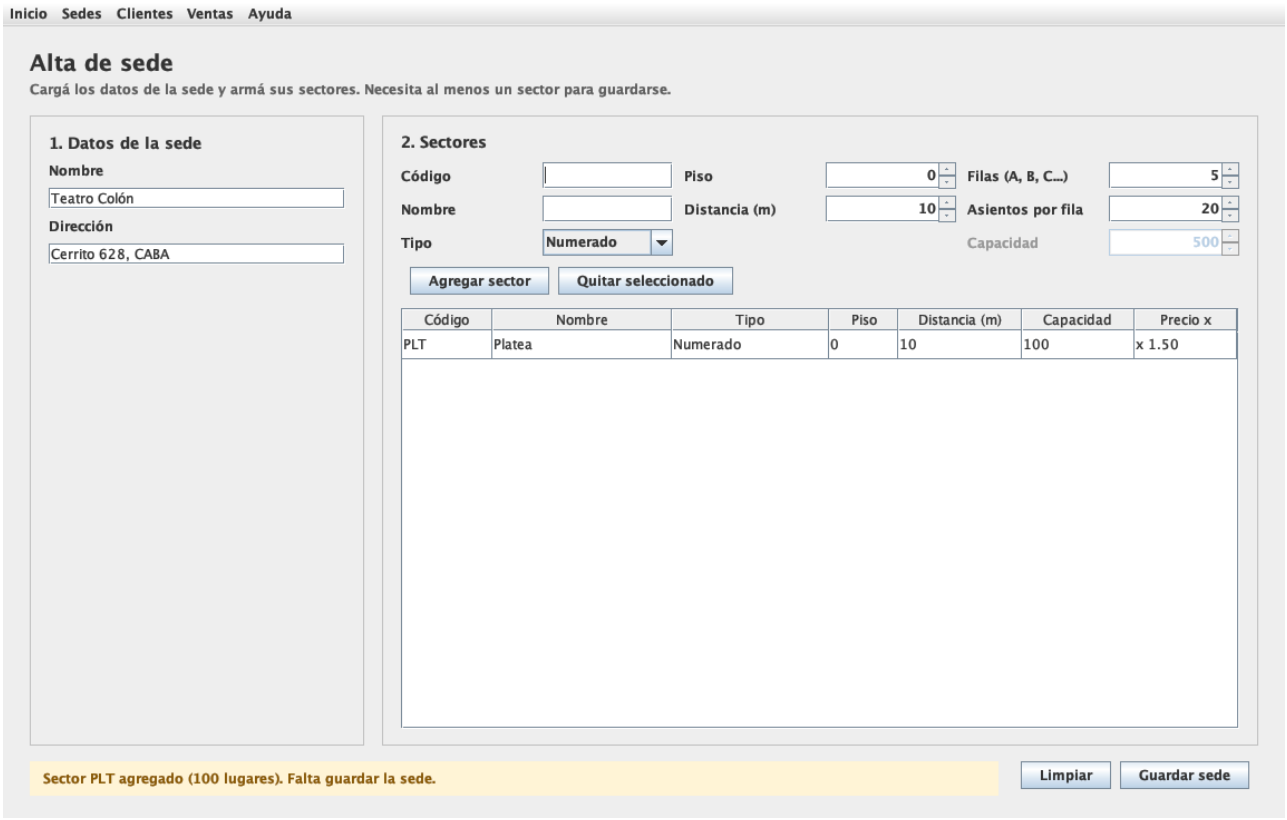


Figura 5. Alta de sede: datos generales y armado de sectores numerados o generales.

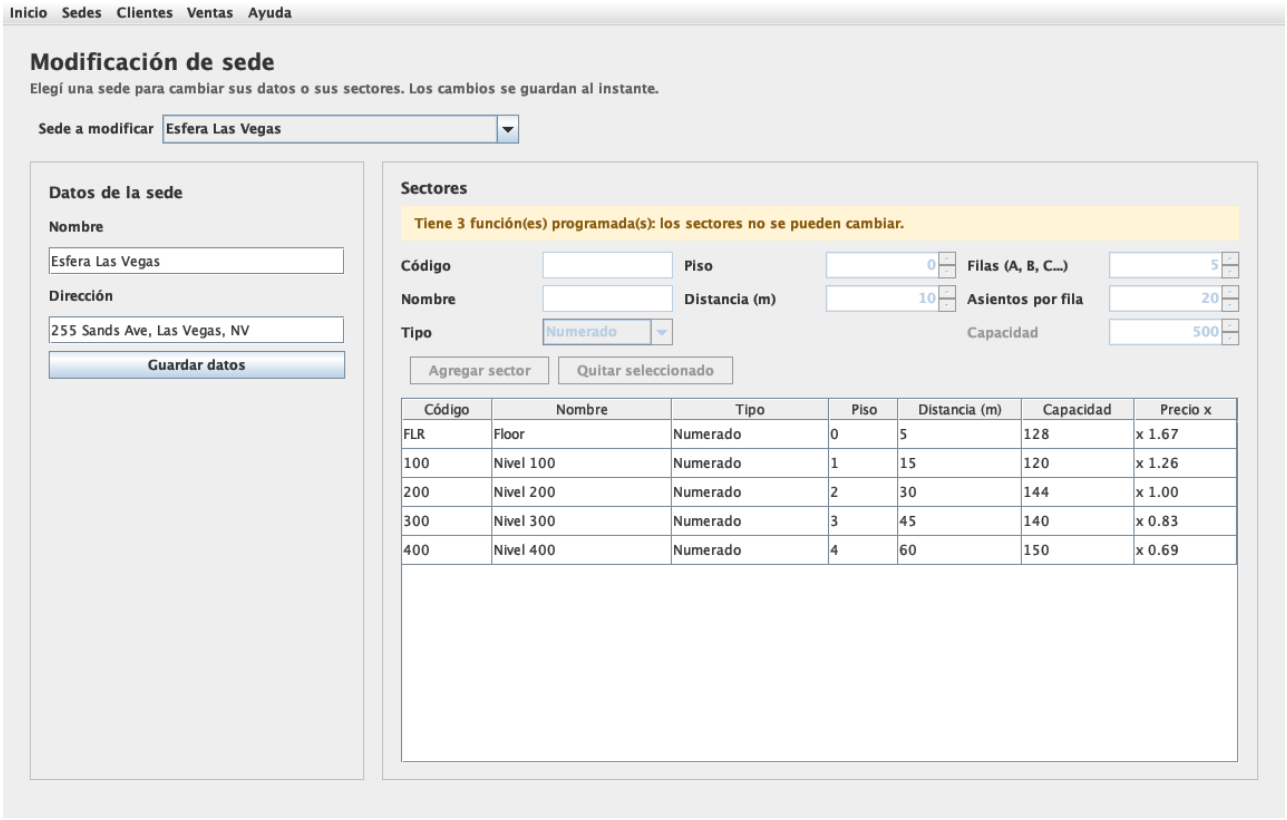


Figura 6. Modificación: una sede con funciones programadas no permite cambiar sus sectores.

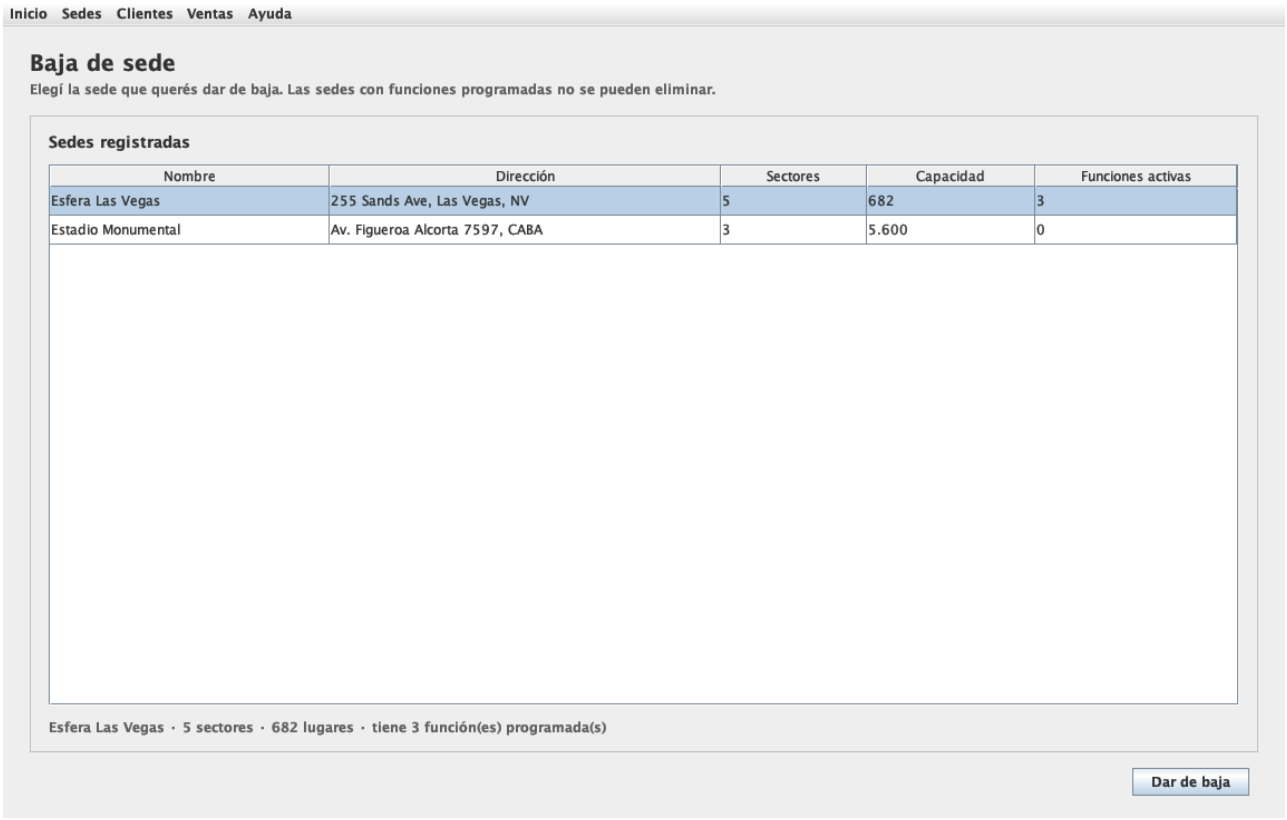


Figura 7. Baja de sede: pide confirmación en un diálogo y no permite eliminar una sede con funciones programadas.

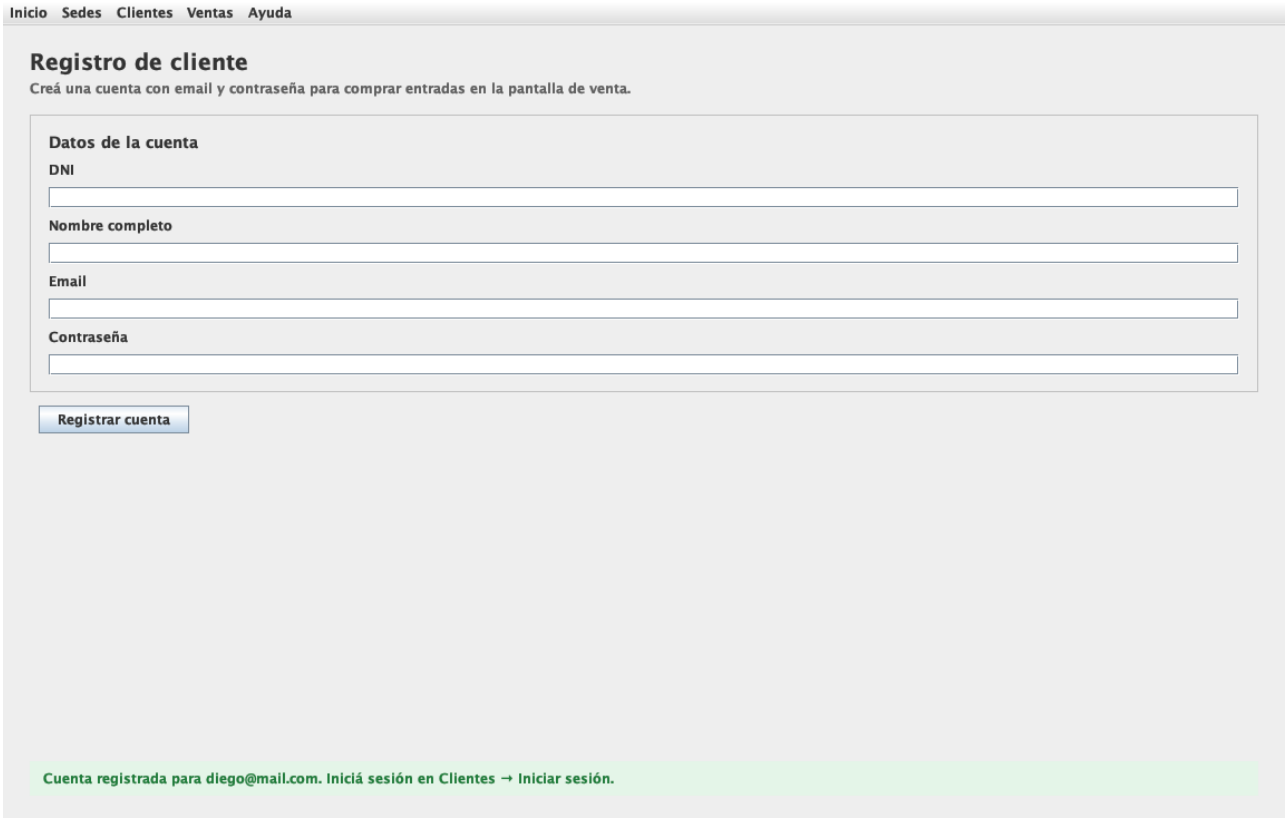


Figura 8. Registro de cliente: alta de cuenta con DNI, nombre, email y contraseña; se guarda en clientes.txt.

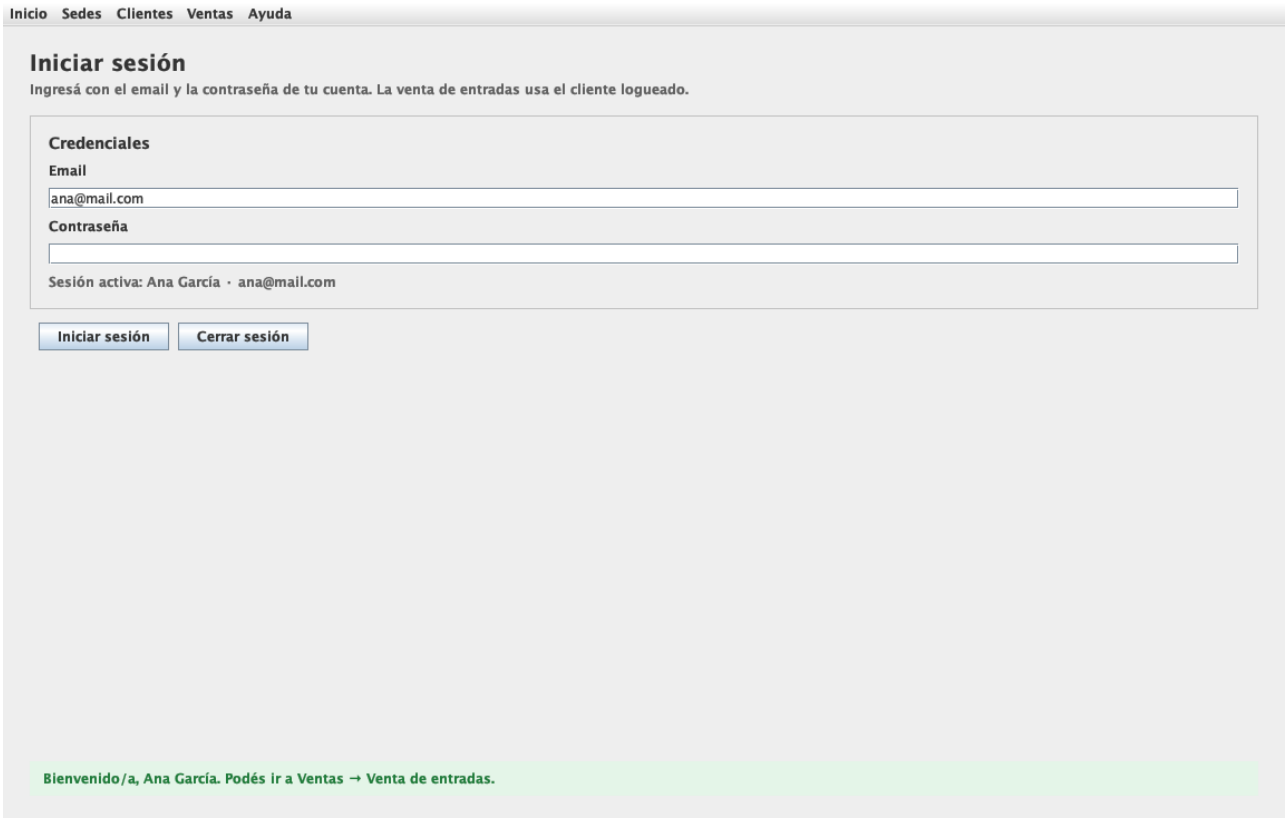


Figura 9. Inicio de sesión: valida email y contraseña y deja al cliente logueado para comprar.

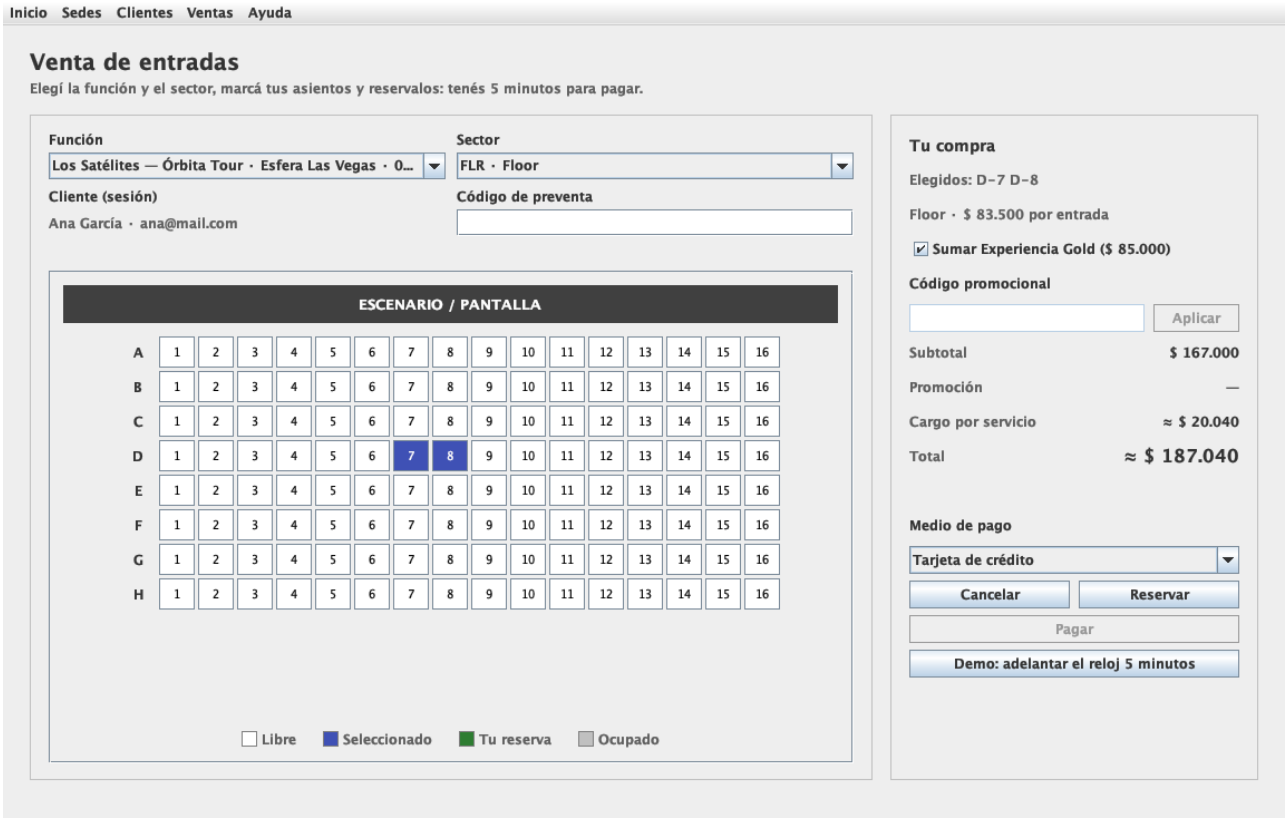


Figura 10. Venta: mapa de asientos con el escenario, selección y resumen estimado.

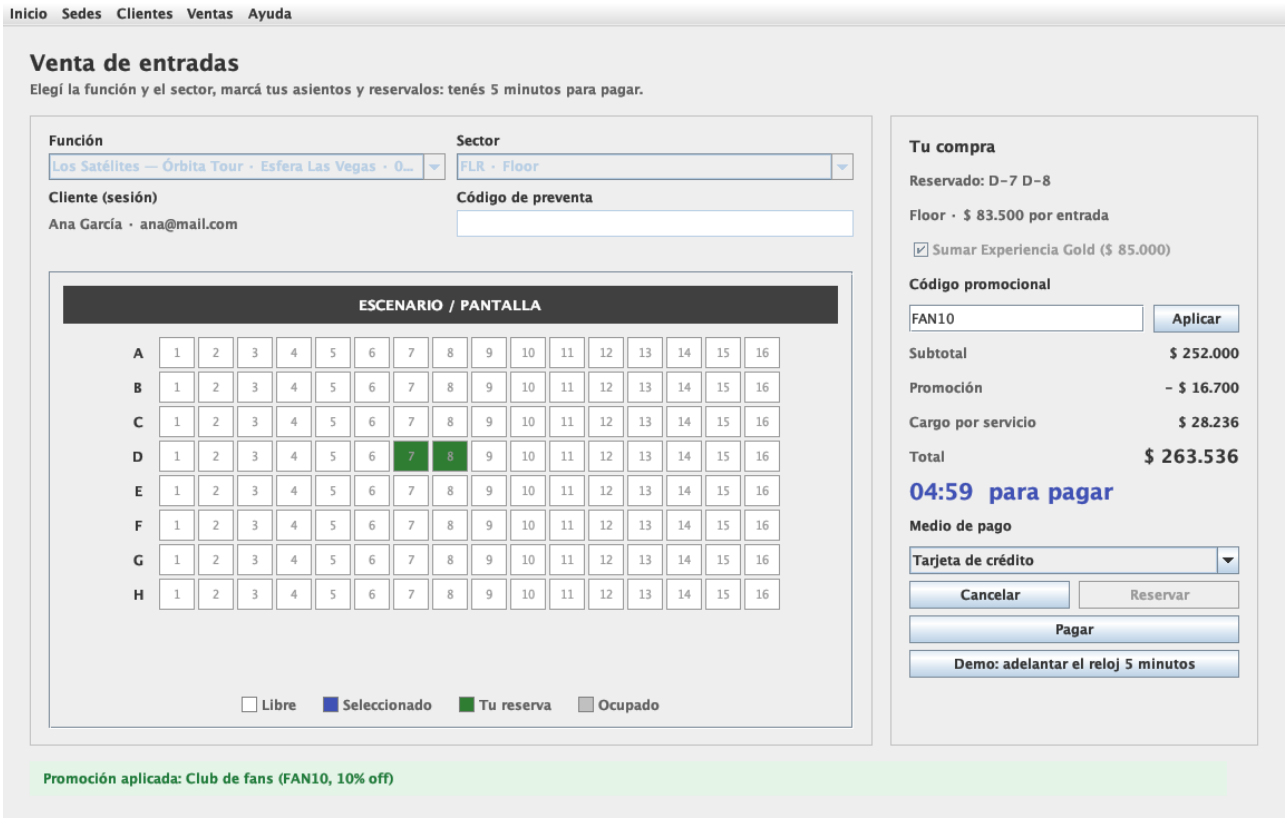


Figura 11. Venta: asientos reservados, cuenta regresiva de 5 minutos y promoción aplicada.

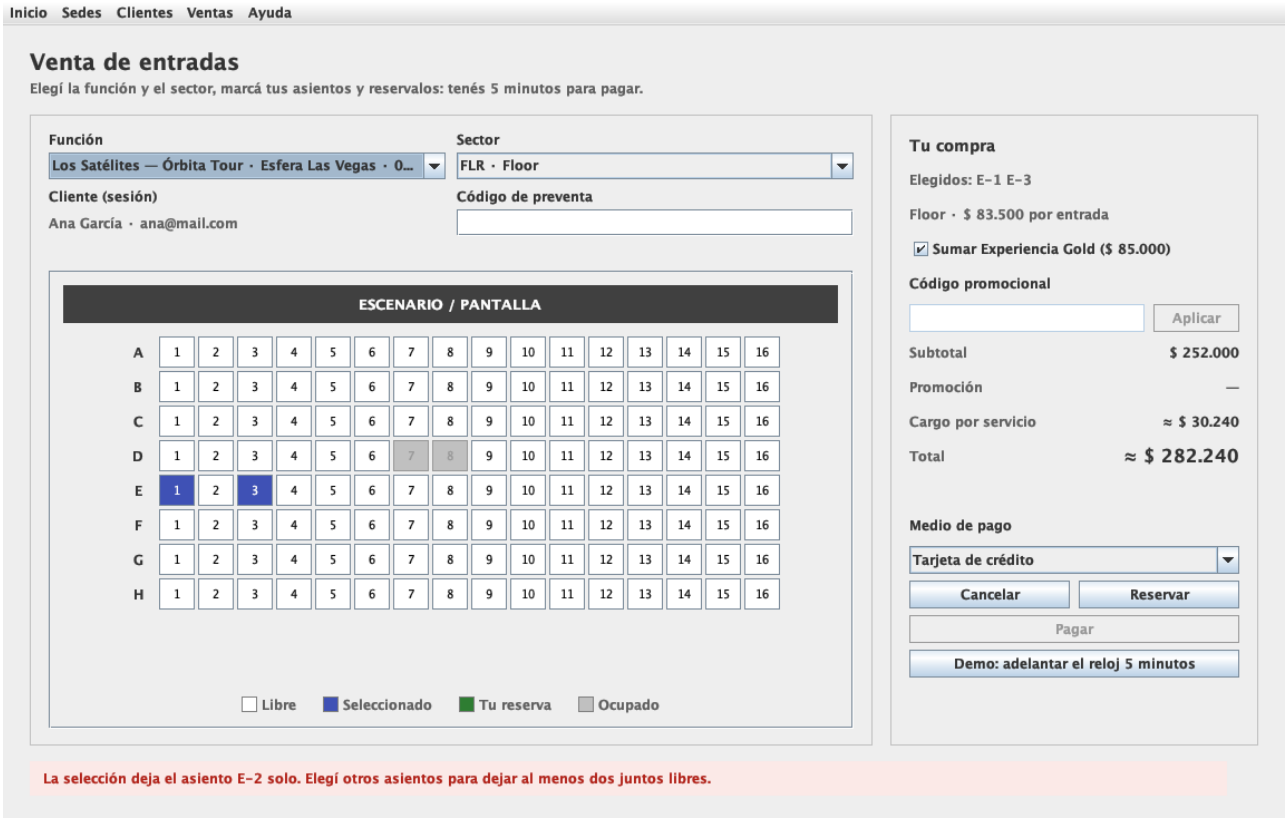


Figura 12. Venta: la regla NoDejarAsientoSuelto rechaza una selección salteada.

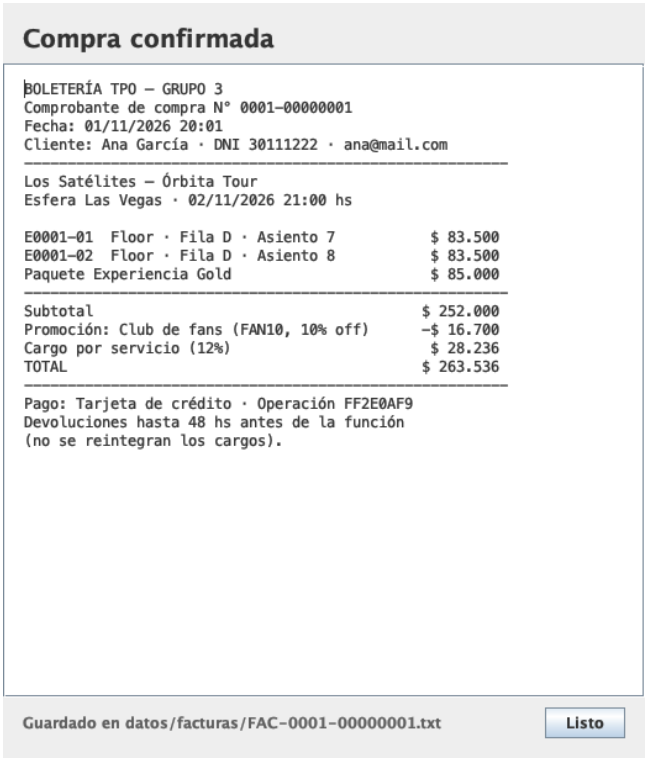


Figura 13. Comprobante emitido al confirmar el pago (diálogo modal).

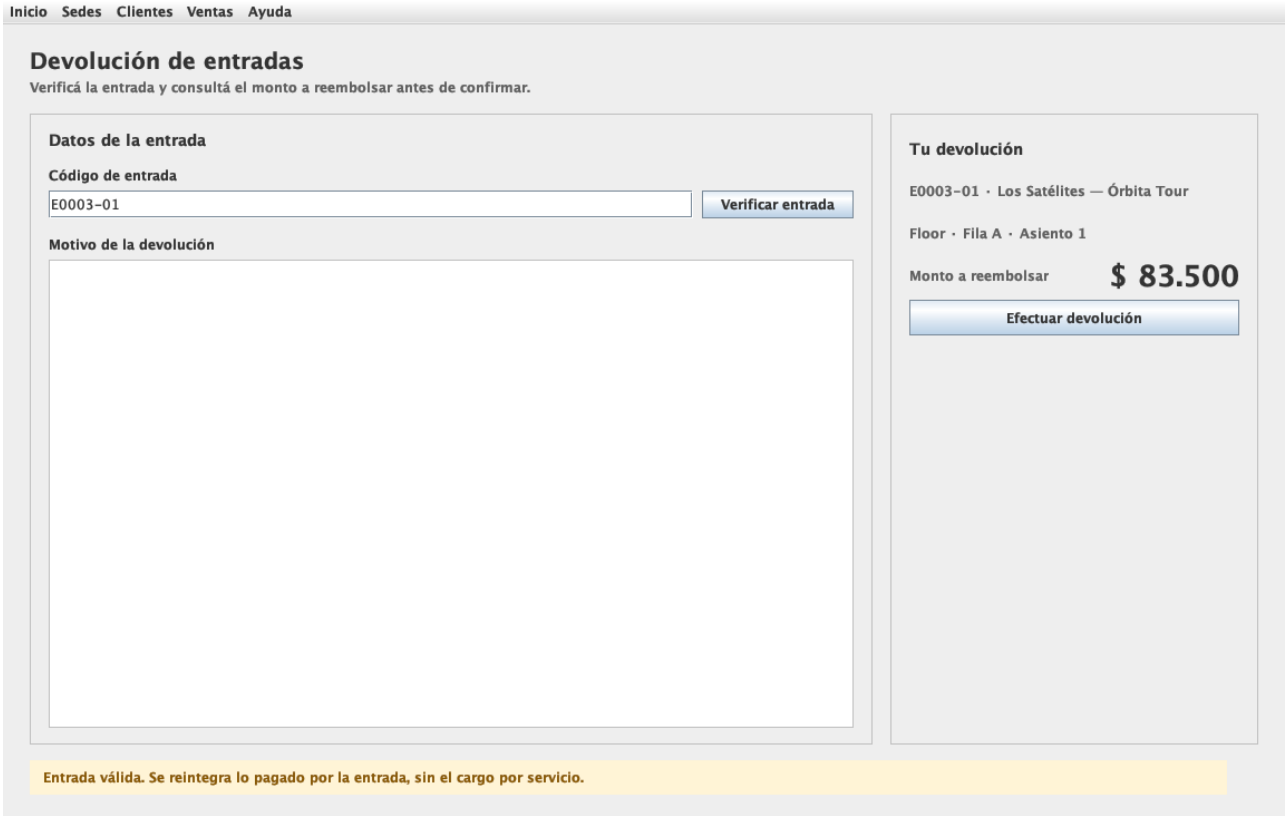


Figura 14. Devolución de entradas: se verifica la entrada por su código y se muestra el monto a reintegrar.